

Authentication Concepts and OAuth 2.0: Beginner Notes

Oct 7, 2026 · @Salike Hassan

At a glance: what kind of thing is each concept?

The most common mix-up is treating these as one list of "login methods". They are five different kinds of thing, and knowing which kind each is removes most of the confusion. Authentication answers *who are you?*; a server that cannot confirm that replies with `401 Unauthorized`.

Concept	What kind of thing it is	Its job in one line
Basic, Digest	Authentication method	Send the username and password, encoded or hashed, with every request
API key	Authentication method	A random string per client, looked up in a database
Session and cookie	Authentication method (stateful)	The server remembers you; a cookie carries the session ID
Bearer	A pattern for sending a token	Whoever holds the token gets access
JWT	A token format	A signed JSON object holding user ID, expiry and claims
Access and refresh tokens	Token roles	Short-lived pass for APIs; long-lived pass to get a new one
OAuth 2.0	Authorization framework	Lets an app act for you with limited access, without your password
OpenID Connect (OIDC)	Identity layer on top of OAuth 2.0	Adds an ID token that says who signed in
SAML	Identity protocol (XML)	Older enterprise sign-in using a signed assertion

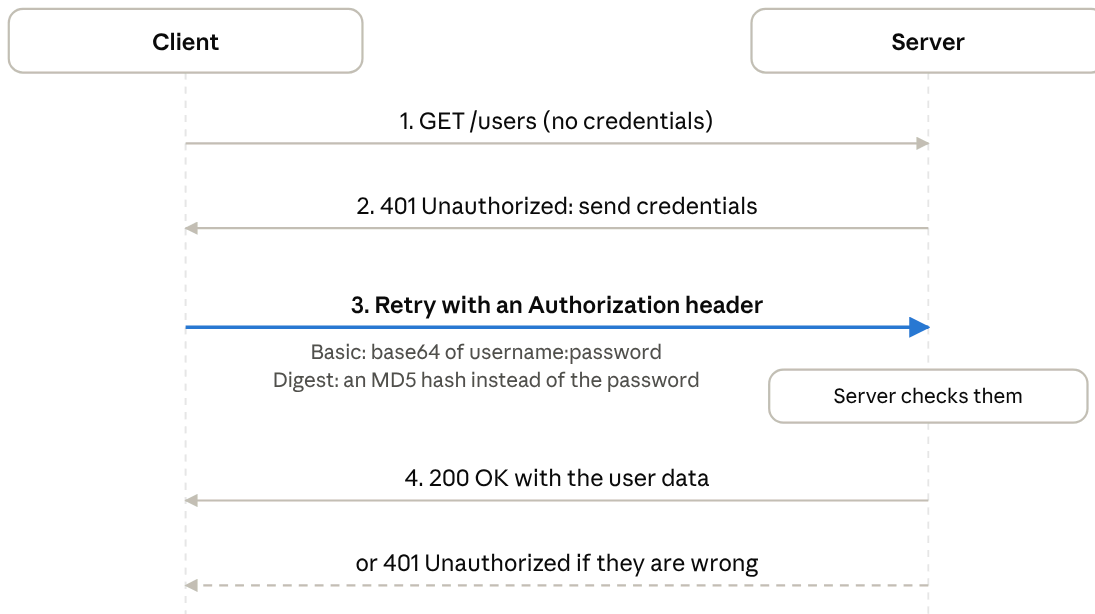
Concept	What kind of thing it is	Its job in one line
Single sign-on (SSO)	User experience pattern	Sign in once, use many apps; built on SAML or OIDC

Two corrections the video makes: JWT is a token *format*, not an authentication method, and OAuth 2.0 is an *authorization* framework, not an authentication method.

1. Basic and Digest authentication

Basic and Digest are the oldest methods: the server challenges the client, and the client then sends its credentials with every request. Both are rarely used in production today outside internal tools.

Basic and Digest: the server challenges, then the client sends credentials



Highlighted arrow: where the credentials travel, on every request.

Basic is only safe inside HTTPS; Digest uses outdated MD5. Both are rare today.

Basic and Digest · challenge and retry

The first request fails with a 401, then the client retries with credentials; Basic and Digest differ only in what goes into the header.

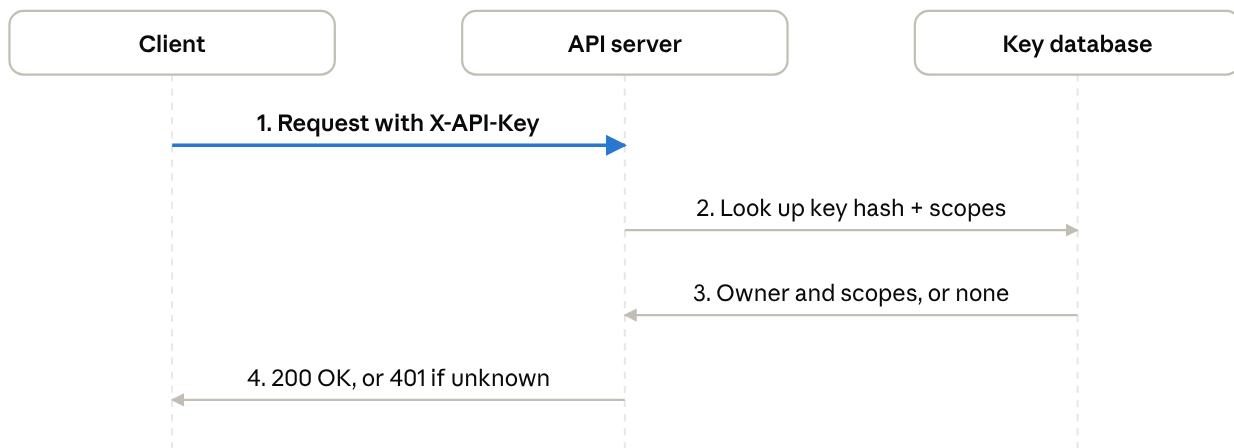
	Basic	Digest
What is sent	Base64 of <code>username:password</code> in the Authorization header	An MD5 hash instead of the plain password
Main weakness	Base64 is easily reversed, so it is only safe inside HTTPS	MD5 hashing is outdated
Status today	Rare; internal tools	Rare; better options exist

The weakness they share: credentials travel on every single request.

2. API keys

The server generates a unique key for each client, and the client sends it with every request, in the Authorization header or an `X-API-Key` header. The server stores each key's hash and its scopes in a database.

API key: every request needs a database lookup



Highlighted arrow: the key travels with every request.

The key is a random string with no embedded information, so the lookup is needed.

A leaked key works for anyone, and it has no built-in expiry.

API key · request and lookup

The server cannot know who owns a key or what it may do without asking the database.

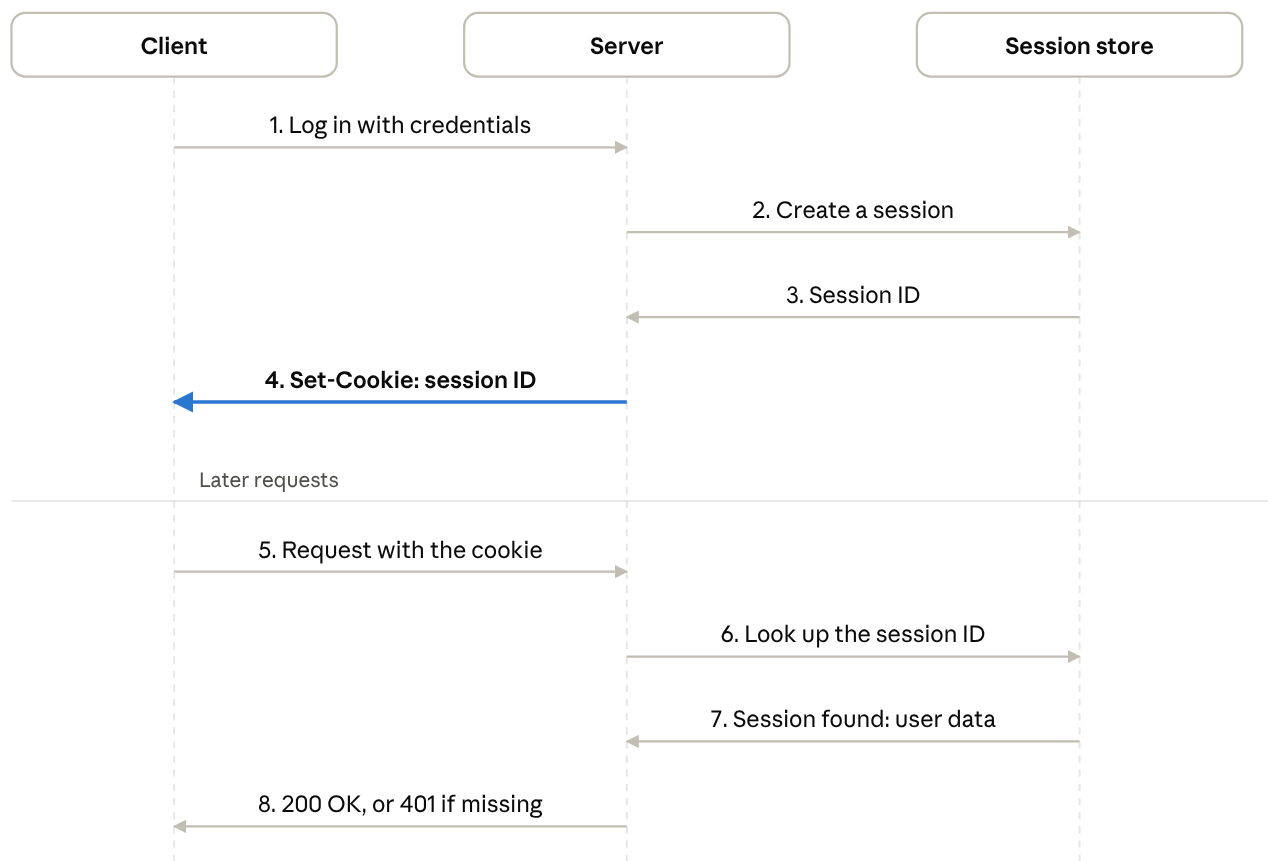
- **Leaks are dangerous:** anyone holding the key can use it.
- **No built-in expiry** unless you build it yourself.

- **No embedded information:** a key is a random string, so the server must look it up to learn who owns it and what it may do. A JWT, by contrast, carries that information inside.

3. Sessions and cookies

After login, the server creates a session in a session store and sets a session cookie on the client. Each later request carries the cookie, and the server looks the session up. This is the traditional web approach.

Sessions: the server remembers you and the cookie carries the ID



Highlighted arrow: the server hands the session ID to the client as a cookie.

Stateful: the server must remember every session in a session store.

Store options: memory (lost on restart), Redis (fast, has expiry), or a SQL database.

session login and later requests

Steps 1 to 4 happen once at login; steps 5 to 8 repeat on every later request.

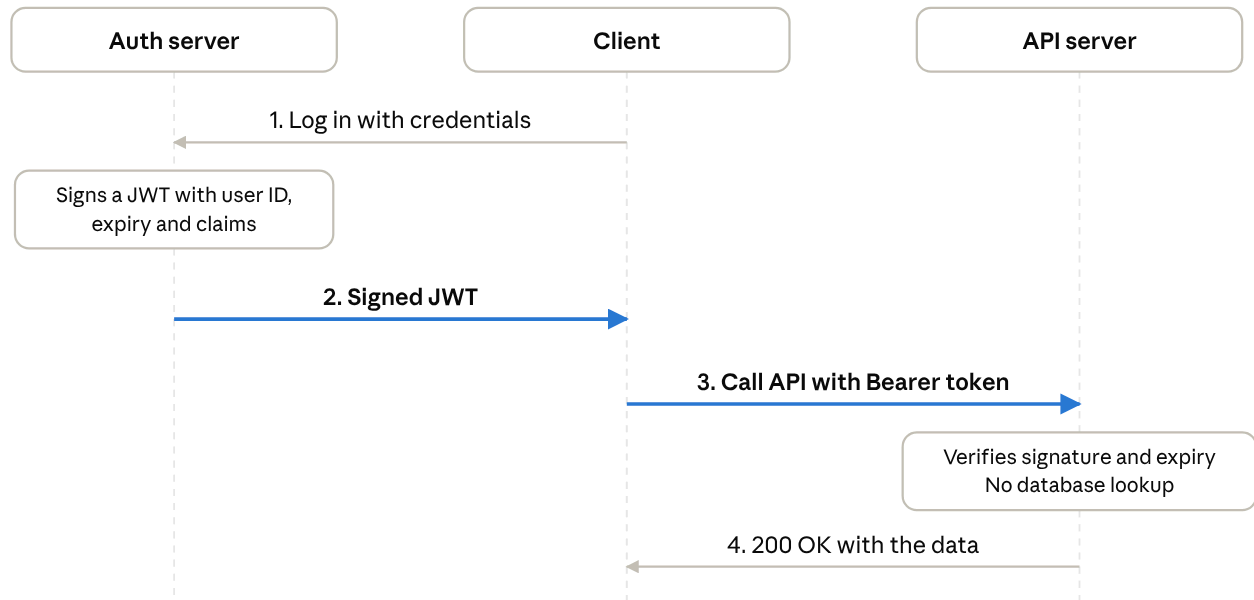
Session store	How it behaves
In memory (a variable)	Fast, but everything is lost when the server restarts or crashes
Redis	Common in production; fast and supports session expiry
SQL database	Durable, a dedicated store for sessions

The catch: sessions are **stateful**, meaning the server must remember them. That works well for traditional web apps but scales less easily for APIs and distributed systems.

4. Bearer tokens and JWT

Modern apps send a token with each request instead of keeping sessions. Two terms get mixed up here, so separate them first.

Term	What it is	Not to be confused with
Bearer	A pattern: whoever holds the token gets access. Sent as <code>Authorization: Bearer <token></code>	A specific token type
JWT (JSON Web Token)	A token format: signed JSON holding user ID or email, expiry time, and claims such as roles and permissions	An authentication method

JWT as a bearer token: the API verifies the signature, no lookup

Highlighted arrows: the JWT is issued, then sent as a bearer token.

Bearer: whoever holds the token gets access. JWT: the signed token format.

Stateless: the API trusts the signature, so it needs no session store.

JWT issued and verified · 4 steps

The API never calls the auth server or a database; it checks the signature and expiry itself.

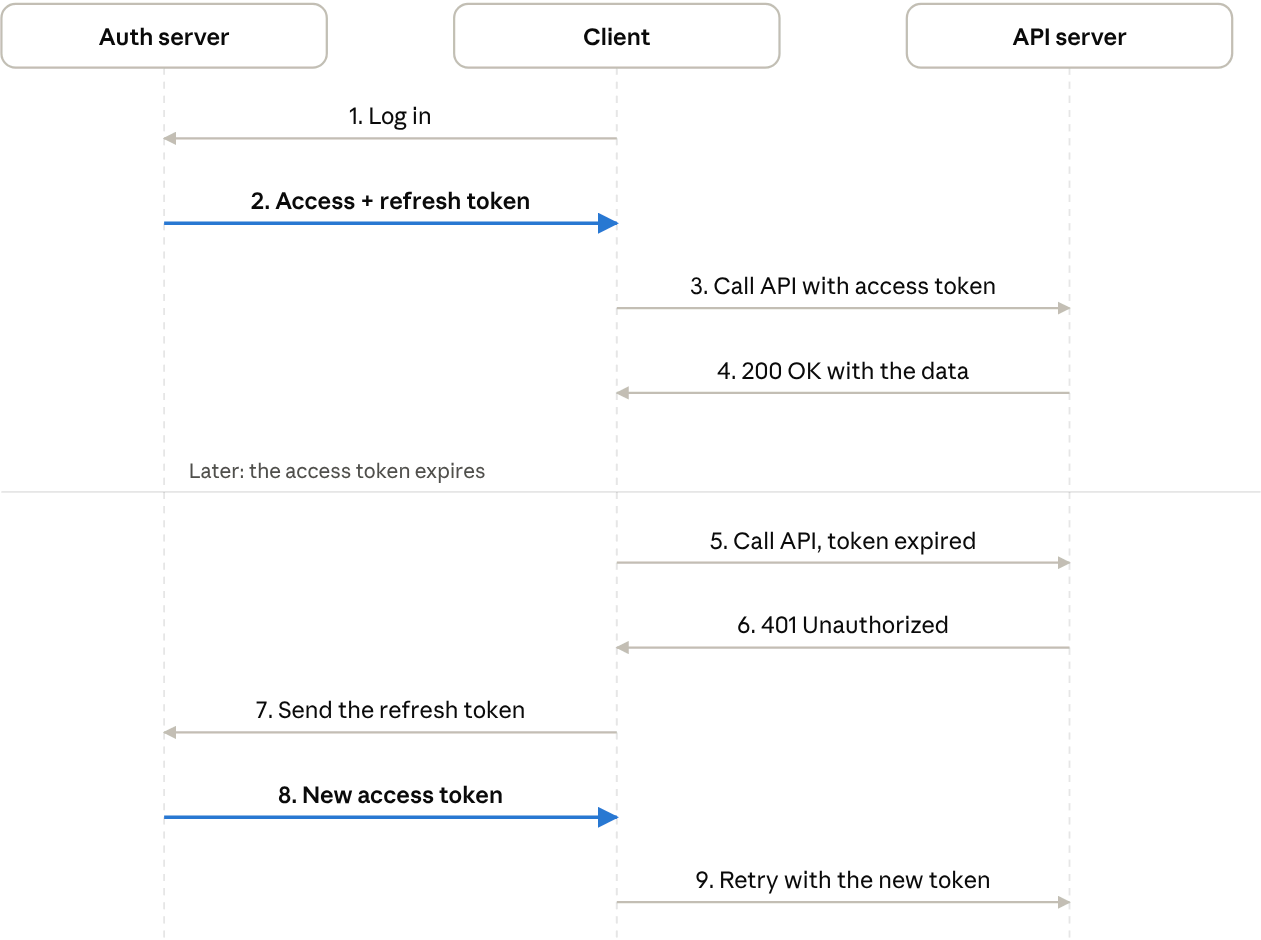
Because a JWT is signed, the server can verify it without any database lookup. That makes it **stateless** and easy to scale, which is the opposite of session-based authentication.

Worth knowing: a JWT is signed, not hidden. Anyone holding it can read its contents, so never put secrets inside one.

5. Access and refresh tokens

Modern systems issue two tokens at login so that a leaked token does little harm: the one used on every call expires fast, and the long-lived one travels rarely.

A refresh token renews an expired access token without a new login



Highlighted arrows: the authorization server issues tokens.
Access token: short-lived (15 minutes to 1 hour), sent on every API call.
Refresh token: long-lived (days or weeks), sent only to the authorization server.
Keep it in an HTTP-only cookie, never in local storage, to guard against XSS.

access and refresh tokens · 9 steps

When the short-lived access token expires, the client trades the refresh token for a new one instead of asking you to log in again.

	Access token	Refresh token
Lifetime	Short: 15 minutes to 1 hour	Long: days or even weeks
Used for	Calling the API on every request	Only getting a new access token

	Access token	Refresh token
Where it goes	The client sends it to the API	Stored by the client; sent only to the authorization server

Store the refresh token in an **HTTP-only cookie**, never in local storage. That protects it from XSS attacks on the client side. Entra ID follows this same two-token pattern.

6. OAuth 2.0

OAuth 2.0 is like giving someone a special key that opens only specific information in another application. You decide who gets access, you never share your password, and you can revoke the key at any time. It replaces the early-internet habit of handing your username and password to another service, which some personal finance tools still do to scrape bank data.

The example: your photos are in **SnapStore**, and you want the printing service **PrintMagic** to print some of them. These are the four roles:

Role	Who it is in the example	Plain meaning
Resource owner	You	Owns the photos
Client	PrintMagic	The app that wants access to the photos
Authorization server	SnapStore's sign-in and consent service (can be an external identity provider)	Handles login and consent, then issues codes and tokens
Resource server	SnapStore's photo storage	Holds the data and accepts access tokens

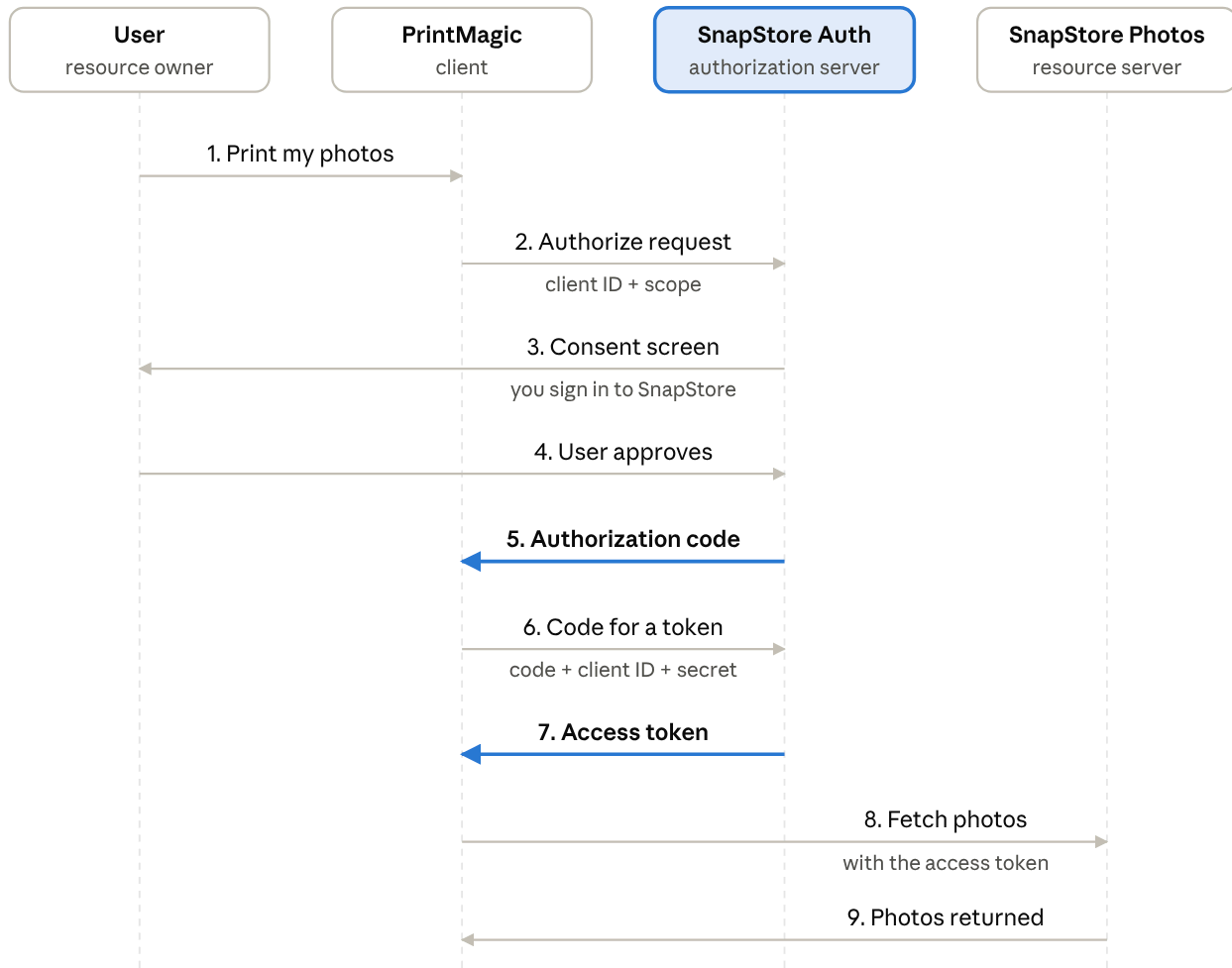
The terms you will see in the flow:

Term	Meaning
Client ID	Identifies the client app (PrintMagic)
Scope	The level of access requested, such as read photos

Term	Meaning
Client secret	A private key shared only between the client and the authorization server
Authorization code	A one-time code returned after you consent
Access token	The pass for the resource server; can expire or be revoked
Refresh token	Gets a new access token without asking you again

In the flow below, your SnapStore password only ever goes to SnapStore. PrintMagic receives a code, swaps it for a token in step 6 directly with the authorization server, and can then fetch only the photos you approved.

OAuth 2.0: PrintMagic gets a token without ever seeing your password



Highlighted arrows: the authorization code and the access token.

Steps 6 and 7 go directly between PrintMagic and the authorization server, using the client secret.

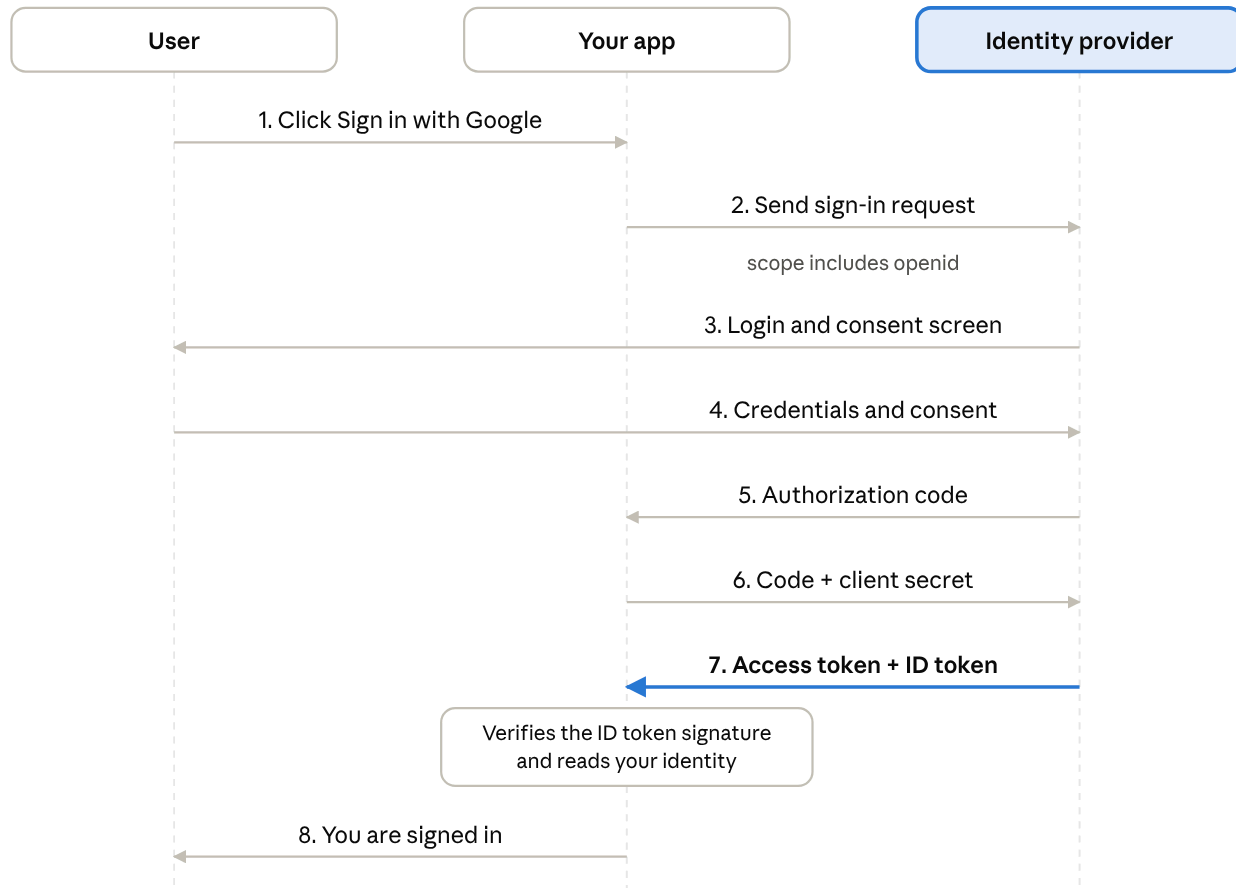
Your SnapStore password is only ever typed at SnapStore (steps 3 and 4).

OAuth 2.0 authorization code flow · 9 steps

This is the diagram from your screenshot, redrawn so you can edit the labels. It is called the authorization code flow because of the code in step 5.

7. OpenID Connect (OIDC)

OIDC adds authentication on top of OAuth 2.0. When you click "Sign in with Google" in an app, the app sends you to the provider's authorization endpoint, you sign in and consent, and the app exchanges the returned code for **two** tokens.

OpenID Connect: the same flow as OAuth 2.0, plus an ID token

Highlighted arrow: the ID token says who signed in; the access token says what the app may do.
OAuth 2.0 alone returns only the access token; OpenID Connect adds the ID token (a JWT).

OpenID Connect sign-in · 8 steps

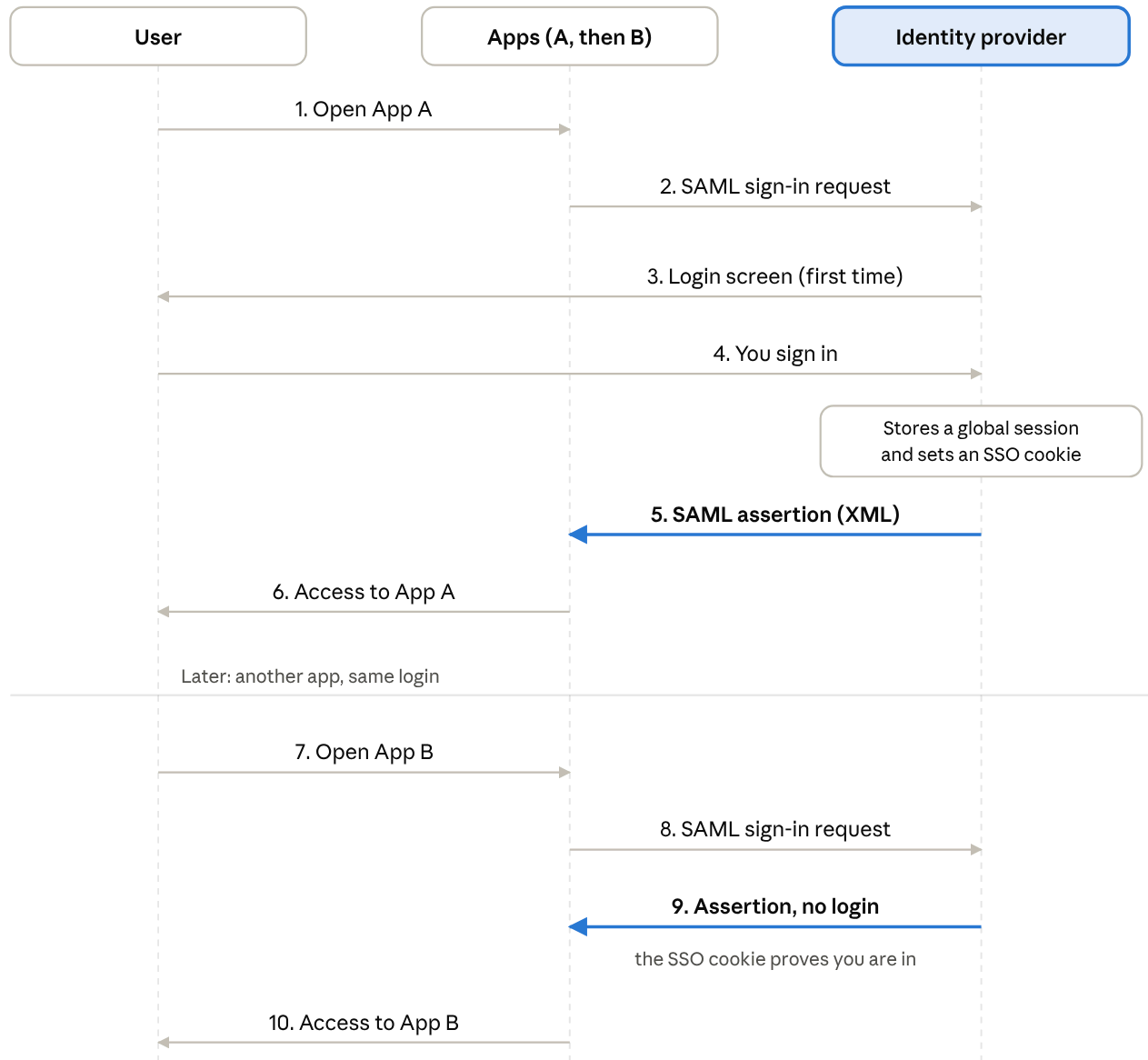
Steps 1 to 6 match the OAuth 2.0 flow; step 7 is where the ID token joins the access token.

	OAuth 2.0	OpenID Connect
Question it answers	What can this app access on my behalf?	Who just signed in?
Token it adds	Access token	ID token (a JWT with your user ID and email or username), alongside the access token
Example	Connect Google Drive so an app can read your files	Sign in with Google
Kind	Authorization framework	Identity layer on OAuth 2.0

The app verifies the ID token's signature, reads your identity from it, and can pass the token to its back end for verification.

8. Single sign-on (SSO) and SAML

SSO is a user experience, not an authentication method: you sign in once at an identity provider and reach many services, such as Gmail, Drive, YouTube and Calendar. After the first sign-in, the identity provider stores a global session and gives your browser an SSO cookie, so other apps do not ask you to log in again.

SSO: sign in once at the identity provider and reach many apps

Highlighted arrows: the identity provider issues an assertion.

SSO is the experience; SAML (XML) or OpenID Connect (ID token) does the work underneath.

SSO with SAML · two apps, one login

The first app triggers a login; the second app reuses the identity provider's session, so you are not asked again.

Under the hood, SSO relies on an identity protocol:

	SAML	OpenID Connect
Message format	XML assertion	JSON; the ID token is a JWT
Age	Older, still widely used	Newer, the most common today
Typical use	Enterprise and legacy systems such as Salesforce and corporate dashboards	Modern web and mobile apps

Authentication is only the first step. Once the system knows *who* you are, authorization decides *what* you may do.

How this maps to your Entra work

Entra ID is an OAuth 2.0 and OpenID Connect provider (it also speaks SAML), so everything above applies directly to your Azure setups. The caller of your secured Function App used the **client credentials flow**: an OAuth 2.0 flow with no user, where the app authenticates as itself.

client credentials flow · your Function App setup

Compare this with the PrintMagic flow: there is no user, no consent screen, no authorization code, and no refresh token.

	Authorization code flow (PrintMagic)	Client credentials flow (your Function App)
User involved?	Yes: signs in and consents	No: the app acts as itself
Sent to the token endpoint	Authorization code, client ID, client secret	Client ID and client secret (or a certificate)
Permission type	Delegated: scopes the user consents to	Application permissions (app roles) granted by an admin
Scope you request	Resource scopes, such as <code>mail.read</code>	The API's Application ID URI plus <code>/.default</code>

	Authorization code flow (PrintMagic)	Client credentials flow (your Function App)
Tokens returned	Access token; refresh token if <code>offline_access</code> is requested; ID token if <code>openid</code> is requested	Access token only; refresh tokens are never granted

How the video's terms line up with what you built:

In the video	In your Function App setup
Client (PrintMagic)	The calling app, represented by its app registration
Client ID and client secret	The Application (client) ID and the secret you generated
Authorization server	Microsoft Entra ID
Resource server	Your Function App, protected by Microsoft authentication (Easy Auth)
Scope	The Application ID URI you exposed, plus <code>/.default</code>
Access token	Sent to the API as <code>Authorization: Bearer <token></code>

Worth checking on your Function App. Microsoft's client credentials page notes that Entra can issue app-only tokens without a `roles` claim, so an API must enforce its own permission checks. The alternative is to require app role assignment, so apps without an assigned role cannot get a token for the API at all. Ask yourself whether your setup does either.

Choosing between methods

No method is best everywhere; each fits a situation and has a known weak spot.

Method	Server stores state?	Good for	Watch out for
Basic, Digest	No	Internal tools only	Credentials on every request; weak encoding or hashing
API key	Yes, a database lookup	Server-to-server access for a known client	A leaked key works for anyone; no built-in expiry
Session and cookie	Yes, a session store	Traditional web apps	Harder to scale for APIs and distributed systems
JWT as a bearer token	No, it verifies the signature	APIs and microservices	Keep it short-lived; never put secrets inside
OAuth 2.0	Authorization server issues tokens	Letting an app act with limited access, for a user or for itself	It is authorization, not login
OpenID Connect	Same, plus an ID token	"Sign in with..." experiences	Needs an identity provider
SSO with SAML or OIDC	Identity provider keeps a global session	Many apps, one login	SSO is an experience, not a method

Check questions

Try each one in your own words before reading the model answer.

Q1. Is a JWT an authentication method? Why or why not?

- Model answer: no. A JWT is a token *format*: a signed JSON object carrying claims such as user ID and expiry. It can be used inside an authentication setup, but it does not itself verify anyone.

Q2. How do sessions and JWTs differ in what the server must remember?

- Model answer: with sessions the server keeps a session store and looks each cookie up (stateful). With a JWT the server just verifies the signature and expiry, so it remembers nothing (stateless).

Q3. In the PrintMagic example, why does PrintMagic never see your SnapStore password?

- Model answer: you sign in and consent directly with SnapStore's authorization server. PrintMagic only receives an authorization code, which it swaps for an access token using its client ID and client secret.

Q4. Name the OAuth 2.0 role of: you, PrintMagic, SnapStore's photo storage, SnapStore's login and consent service.

- Model answer: resource owner, client, resource server, authorization server.

Q5. What does OpenID Connect add to OAuth 2.0?

- Model answer: authentication. It adds an ID token (a JWT with your identity) next to the access token, so the app learns *who* signed in, not just what it may access.

Q6. Your secured Function App was called by another app with no user present. Why was there no consent screen and no refresh token?

- Model answer: it used the client credentials flow, where the app authenticates as itself with its client ID and secret. No user means nobody to consent, and the app simply requests a new access token when the old one expires.

References

I opened each Microsoft page below to check it. Every flow in this document is covered there with Entra's real endpoints and parameters.

Resource	What it covers	Use it for
OAuth 2.0 authorization code flow	The PrintMagic-style flow on Entra, including client secret redemption, ID tokens and refresh tokens	Section 6 in depth
OAuth 2.0 client credentials flow	App-only tokens, the <code>/.default</code> scope, application permissions	Your Function App setup
OpenID Connect on the Microsoft identity platform	ID tokens, the sign-in request, validating tokens, sign-out	Section 7 in depth

The notes are based on the two video transcripts you shared: the OAuth 2 explainer and the seven authentication concepts video. I do not have their links; paste them and I will add them here.